

# Technical White Paper: Forward Automatic Differentiation in faust-rs

OpenAI Codex

2026-04-24

## Forward Automatic Differentiation in **faust-rs**

### Executive Summary

**faust-rs** now exposes a native forward automatic differentiation primitive, `fad(expr, seed)`, inside the compiler pipeline. This turns differentiation into a compile-time symbolic transform rather than a dynamic runtime graph.

The practical result is that a Faust DSP program can now emit both its primal signal and exact local derivatives with respect to explicit seed parameters, while keeping the normal static Faust compilation model:

- parsing and evaluation stay symbolic,
- propagation builds a differentiated signal graph,
- FIR lowering and backends emit ordinary DSP code,
- recursion is handled without requiring an external ML runtime.

This is not reverse-mode machine learning infrastructure retrofitted onto DSP. It is a forward-mode differentiable extension of the Faust signal language, implemented directly in the Rust compiler.

### 1. Overview

**faust-rs** now exposes a forward-mode automatic differentiation primitive, `fad(expr, seed)`, inside the Faust language accepted by the Rust compiler. It lets a DSP author ask for the value of an expression and its derivative with respect to one or more explicit seed signals, without manually rewriting the signal graph into dual form.

At a high level:

- `expr` is the DSP expression to differentiate.
- `seed` selects the variable or variables with respect to which the derivative is taken.
- the result is a multi-output DSP expression:
  - first the primal value of `expr`,
  - then one tangent output per seed.

The primitive is deliberately explicit. **faust-rs** does not implicitly differentiate with respect to “all controls in scope”; instead, the seed expression determines the differentiation variables. This makes the primitive predictable, composable, and much easier to reason about in recursive code.

The current implementation focuses on **forward AD**. Reverse AD (**rad**) is not part of the supported compilation path yet.

This note summarizes:

1. the surface semantics of `fad(expr, seed)`,
2. the compiler pipeline used to implement it,
3. the recursion model and why it matters,
4. the supported boundaries of the feature,
5. practical DSP usage patterns with examples.

## 2. Surface Semantics

### 2.1 Basic rule

Conceptually:

```
fad(expr, seed) = [ expr, d(expr)/d(seed_0), d(expr)/d(seed_1), ... ]
```

If `seed` is a single signal, `fad` produces two outputs:

- primal,
- tangent.

If `seed` is a tuple-like parallel expression, `fad` produces one tangent per seed lane. For example:

```
process = fad(x * y, (x, y));
```

returns three outputs:

1.  $x * y$
2.  $d(x*y)/dx = y$
3.  $d(x*y)/dy = x$

### 2.2 Tangent extraction

Because `fad` returns the primal first, a common pattern is to select only the derivative lane:

```
grad = fad(loss, param) : !, _;
```

This keeps the tangent and discards the primal. In the current output ordering, `!`, `_` is the standard “tangent-only” projection for a single-seed `fad`.

### 2.3 Multi-seed behavior

Multi-seed `fad` is observable in output order:

```
f = hslider("freq", 1000, 50, 4000, 1);
q = hslider("q", 1.0, 0.1, 10.0, 0.01);
x = no.noise;

process = fad(x : fi.resonlp(f, q, 1.0), (f, q));
```

This produces:

1. the filter output,

2. the derivative with respect to `f`,
3. the derivative with respect to `q`.

The implementation preserves this ordering throughout propagation and code generation.

### 3. Why `fad` Matters in DSP

Forward AD is useful whenever a DSP patch needs local sensitivity information. Typical examples are:

- gradient monitoring for adaptive algorithms,
- parameter estimation,
- host-driven optimization,
- differentiable control analysis,
- recursive adaptive updates where the gradient of a loss depends on the previous state.

In a Faust setting, `fad` is attractive because it operates directly on the signal graph. A user does not need to manually define derivative versions of `sin`, `pow`, `delay`, or recursive projections; the compiler applies the chain rule structurally.

This makes small differentiable DSP programs much easier to write and maintain than hand-written dual graphs.

### 4. How `fad` Is Implemented

The implementation follows the normal `faust-rs` pipeline:

```
parse -> boxes -> eval -> propagate -> normalize -> transform -> fir -> backend
```

#### 4.1 Parser and box layer

At parse time, `fad(expr, seed)` is recognized as a dedicated box node. The relevant surface is in:

- `crates/boxes/src/builder.rs`
- `crates/boxes/src/matcher.rs`

The parser builds a `ForwardAD(expr, seed)` box wrapper rather than expanding the derivative immediately. That is important because the rest of the compiler still needs to flatten modules, substitutions, metadata, and recursive aliases before signal-level differentiation is safe.

#### 4.2 Evaluation

The evaluator preserves the `ForwardAD` wrapper instead of trying to interpret it as an ordinary computation. This keeps `fad` visible until signal propagation.

The same area also contains the recursion alias preservation work that made patterns such as:

```
state = next ~ _;
prev  = state;
next  = saturate(prev + input);
```

semantically usable inside the Rust compiler. That matters because many useful AD examples refer to the previous recursive state by name.

### 4.3 Propagation and signal differentiation

The actual AD transform lives in:

- crates/propagate/src/forward\_ad.rs
- crates/propagate/src/lib.rs

This is the key stage. Once the box graph has been lowered to signals, `generate_fad_signals_multi(...)` invokes a `ForwardADTransform` that traverses the signal DAG and produces a bundle:

```
[primal, tangent_0, tangent_1, ... tangent_n]
```

for every visited node.

The transform is memoized. Shared sub-expressions are differentiated once and reused, which keeps the derivative graph linear in the size of the original DAG.

### 4.4 Multi-lane dual representation

Earlier prototypes effectively ran one differentiator per seed. The current model carries all tangents together in one bundle:

```
Dual {  
    primal,  
    tangents[0..N)  
}
```

That is especially important for recursive code. Instead of duplicating a recursive shadow for each seed, the transform now builds one interleaved recursive carrier containing:

```
[primal, d/ds0, d/ds1, ...]
```

This reduces redundant state, improves generated code, and keeps tangent order stable.

### 4.5 FIR and backend code generation

After propagation, `fad` no longer exists as a source primitive. It has become an ordinary expanded multi-output signal graph. The downstream FIR lowering and the C, C++, interpreter, Cranelift, and Wasm backends just see extra outputs and recursive state lanes.

That design is useful because it keeps AD mostly localized to the propagation phase. The backend does not need a dedicated differentiation pass.

## 5. Recursion Support

Recursion is where AD becomes technically interesting. A purely feed-forward transform is straightforward; recursive DSP requires careful handling of delayed self-reference.

### 5.1 Local recursive gradients

The following pattern is supported:

```
pan = step ~ _ with {  
    target = hslider("Target", 0, -1, 1, 0.01);  
    lr = hslider("LR", 0.05, 0, 1, 0.001);
```

```

step(prev) = prev - lr * grad with {
  loss = (prev - target) ^ 2;
  grad = fad(loss, prev) : !, _;
};
};

process = pan;

```

This is significant because the derivative is consumed locally inside the recursive branch. The compiler now supports that by switching from the original “expand after recursion” model to an **augmented-state recursion** model when a recursive branch consumes **fad** outputs immediately.

## 5.2 What is still out of scope

The implementation is strong on several recursive families:

- self-recursive state,
- nested recursion,
- mutual recursion,
- multi-output recursion,
- multi-seed recursive differentiation.

However, it is not yet a universal differentiable extension for every Faust construct. In particular:

- **rad(...)** is not supported in the propagation path,
- some external recursive AD feedback patterns still require explicit support work,
- discrete/non-differentiable primitives intentionally fall back to zero tangents.

## 6. Supported Signal Families

The current forward AD implementation covers the signal families most relevant to DSP:

- arithmetic and algebraic nodes,
- standard transcendentals,
- **pow**, **atan2**, **min/max**, **fmod**, **remainder**,
- delays,
- recursive projections,
- explicit seeds with duplicates or multiple lanes,
- read-only table and waveform reads.

Table reads deserve a special note. **faust-rs** now differentiates read-only **rdtable** and waveform lookups with respect to the index using a symmetric finite-difference slope:

$$d(\text{table}[i]) / di \approx (\text{table}[i + 1] - \text{table}[i - 1]) / 2$$

This is practical for modulation, wavetable indexing, and lookup-based models.

The main intentional “zero tangent” boundaries remain:

- buttons and checkboxes,
- integer-only and bitwise operations,
- casts that destroy differentiable structure,

- mutable table writes and broader effectful memory updates.

## 7. Demonstration DSP Examples

### 7.1 Single-parameter gain sensitivity

This is the smallest useful example: measure how the output changes with respect to a gain control.

```
os = library("oscillators.lib");

gain = hslider("gain", 0.5, 0, 2, 0.01);
sig = gain * os.osc(220);

process = fad(sig, gain);
```

Outputs:

1. `gain * osc(220)`
2. `osc(220)`

This is a direct sanity check: the derivative of `gain * x` with respect to `gain` is exactly `x`.

### 7.2 Two-parameter filter sensitivity

This example exposes both the primal filter output and the sensitivities of the filter with respect to frequency and resonance:

```
fi = library("filters.lib");
no = library("noises.lib");

f = hslider("freq", 1000, 50, 5000, 1);
q = hslider("q", 1.0, 0.1, 10.0, 0.01);
x = no.noise;

process = fad(x : fi.resonlp(f, q, 1.0), (f, q));
```

Outputs:

1. filter output,
2.  $d \text{ output} / d f$ ,
3.  $d \text{ output} / d q$ .

This is a good analysis patch when tuning a filter interactively.

### 7.3 Recursive local gradient update

The next example uses `fad` inside a recursive state update:

```
target = hslider("Target", 0, -1, 1, 0.01);
lr      = hslider("LR", 0.05, 0, 1, 0.001);

state = step ~ _ with {
  step(prev) = prev - lr * grad with {
    loss = (prev - target) ^ 2;
```

```

    grad = fad(loss, prev) : !, _;
};
};

```

```
process = state;
```

This is not just a toy. It demonstrates that **faust-rs** can differentiate a loss with respect to the previous recursive state and immediately use that gradient in the same recursive branch.

## 7.4 Host-driven optimization

For larger adaptive examples, a practical pattern is to keep the gradient inside the DSP but close the optimization loop in the host. The repository contains examples of this style:

- tests/corpus/auto\_chorus\_stereo\_fad\_host.dsp
- tests/corpus/auto\_wah\_fad\_host.dsp

The DSP computes:

- primal audio,
- loss,
- one or more gradients,

and the host updates the control parameters externally. This is currently the cleanest strategy for complex adaptive effects.

## 8. Current Limits and Practical Guidance

For day-to-day use, the following guidelines are accurate:

- use `fad(expr, seed)` with explicit seed variables,
- prefer the explicit parallel seed form `(a, b, c)` when differentiating with respect to multiple parameters,
- use `: !, _` to extract the tangent for a single-seed `fad`,
- expect robust behavior on feed-forward graphs and on the recursive families already covered by the corpus,
- use host-driven optimization for larger adaptive systems when the full update loop would otherwise become compiler-sensitive.

The most important non-goal to keep in mind is that **fad** is not yet “differentiate all Faust code automatically”. It is a strong, well-tested forward AD primitive over an explicitly supported differentiable subset of Faust.

## 9. Conclusion

**fad** gives **faust-rs** a meaningful differentiable DSP capability inside the language itself. The primitive is explicit, compositional, and deeply integrated with the Rust compiler pipeline:

- parser and boxes preserve the AD node,
- evaluation keeps it structural,
- propagation performs the actual signal differentiation,
- recursion is handled through augmented-state lowering and unified tangent lanes,
- backends consume the result as ordinary expanded DSP outputs.

That makes `fad` useful in two complementary ways:

1. as an analysis tool, to inspect local sensitivities of a DSP graph,
2. as a building block for adaptive or optimization-driven audio systems.

The implementation is already broad enough to support serious experimentation, especially on feed-forward graphs, recursive local gradient patterns, and host-driven adaptive effects. The remaining work is mostly about extending the supported differentiable subset further, rather than proving the core model.